

Introdução a Algoritmos

Aula 9 – Vetores

Raoni F. S. Teixeira · 1s/2026

Objetivos desta aula

Ao final desta aula, você deverá ser capaz de:

1. Declarar e acessar vetores, explicando por que o índice começa em zero e o que ocorre ao acessar além dos limites;
2. Aplicar as três perguntas de um laço a vetores, identificando o índice como “o que muda” que seleciona dados diferentes;
3. Implementar os padrões fundamentais sobre vetores: leitura, soma/média, máximo/mínimo, busca linear e contagem condicional;
4. Explicar por que passar um vetor para uma função é diferente de passar um int, e por que a função pode modificar o vetor original.

1 Um problema que a Aula 5 não consegue resolver

Na Aula 5 aprendemos a repetir instruções com `while` e `for`. Com isso, conseguimos calcular a média de qualquer quantidade de notas:

```
float soma = 0;
int i = 0;
while (i < n) {
    float x;
    scanf("%f", &x);
    soma += x;
    i++;
}
float media = soma / n;
printf("Media: %.2f\n", media);
```

Aplicando as três perguntas do laço: o que se repete é a leitura e acumulação; repete-se `n` vezes; o que muda é o valor lido e o acumulador `soma`.

Mas agora considere uma pergunta diferente: **quantas notas estão acima da média?** Para responder, precisamos primeiro calcular a média — e depois comparar cada nota com ela. Mas as notas já foram lidas e descartadas. A variável `x` guardou cada nota por uma fração de segundo antes de ser sobrescrita pela próxima.

Não há como resolver esse problema com o que sabemos até agora. Para comparar cada nota com a média, precisaríamos ler tudo duas vezes — o que pode ser impossível — ou guardar todos os valores simultaneamente na memória.

É exatamente isso que um **vetor** faz.

2 O que é um vetor

Definição — Vetor

Um vetor (ou **array**) é uma sequência de variáveis do **mesmo tipo**, armazenadas em posições **contíguas** de memória, acessadas por um **índice** inteiro. Em C, o índice começa em zero.

Lembre-se da peça “dados” apresentada na Aula 1: a variável era sua forma mais simples, um único valor. O vetor é a mesma peça em outro nível — vários valores do mesmo tipo, organizados por índice.

A declaração reserva espaço para n elementos do tipo indicado:

```
float notas[5]; /* 5 variáveis float, uma ao lado da outra na memória */
int v[10];      /* 10 variáveis int */
```

Cada elemento é acessado pelo nome do vetor seguido do índice entre colchetes:

```
notas[0] = 7.5; /* primeiro elemento */
notas[1] = 8.0; /* segundo elemento */
notas[4] = 6.0; /* quinto (e ultimo) */
```

O índice pode ser qualquer expressão inteira — inclusive uma variável de controle de laço. É essa combinação que torna vetores e laços inseparáveis.

3 Vetores na memória

Um vetor não é um tipo misterioso — é apenas um bloco contíguo de bytes, como aprendemos na Aula 8. Para `float notas[5]`, o compilador reserva $5 \times 4 = 20$ bytes consecutivos:

Índice	[0]	[1]	[2]	[3]	[4]
Endereço	1000	1004	1008	1012	1016
Valor	7.5	8.0	?	?	6.0

Cada `float` ocupa 4 bytes; elementos consecutivos distam 4 bytes entre si. Os elementos [2] e [3] ainda não foram escritos — contêm lixo.

O nome `notas` sozinho, sem índice, representa o endereço do primeiro elemento — exatamente como um ponteiro. De fato, `notas[i]` e `*(notas + i)` são expressões equivalentes em C: ambas acessam o elemento na posição i .

Isso tem uma consequência direta que veremos na Seção 6: passar um vetor para uma função é passar o endereço do primeiro elemento. A função acessa a memória original — não uma cópia.

4 As três perguntas aplicadas a vetores

As três perguntas da Aula 5 continuam válidas, mas ganham uma dimensão nova quando trabalhamos com vetores:

1. **O que se repete?** — a operação a executar em cada elemento.
2. **Quantas vezes?** — o número de elementos do vetor (n).
3. **O que muda?** — o **índice** que seleciona qual elemento está sendo processado.

A terceira resposta é o que diferencia laços sobre vetores dos laços da Aula 5: antes, o laço repetia a mesma operação sobre dados efêmeros; agora, o índice seleciona dados **diferentes que já estão na memória**. O laço não apenas repete — ele **percorre**.

5 Padrões fundamentais

Quase toda operação sobre vetores é uma instância do mesmo esqueleto:

```
for (int i = 0; i < n; i++) {  
    /* operacao com v[i] */  
}
```

A seguir, os padrões mais frequentes — todos variações desse esqueleto.

5.1 Leitura e impressão

```
/* Leitura */  
for (int i = 0; i < n; i++) {  
    scanf("%f", &v[i]);  
}  
  
/* Impressao */  
for (int i = 0; i < n; i++) {  
    printf("v[%d] = %.2f\n", i, v[i]);  
}
```

Note o `&v[i]` no `scanf`: precisamos do endereço do elemento, assim como precisávamos do endereço de qualquer variável simples.

5.2 Soma e média

```
float soma = 0;  
for (int i = 0; i < n; i++) {  
    soma += v[i];  
}  
float media = soma / n;
```

Este é o mesmo padrão de acumulação da Aula 5 — mas agora os dados estão guardados no vetor, não sendo lidos na hora. Isso permite percorrer o vetor uma segunda vez depois de calcular a média.

5.3 Máximo e mínimo

```
float maior = v[0]; /* inicializa com o primeiro elemento — regra da Aula 6 */  
for (int i = 1; i < n; i++) {  
    if (v[i] > maior)  
        maior = v[i];  
}
```

O laço começa em $i = 1$ porque $v[0]$ já foi usado na inicialização.

5.4 Busca linear

```
float alvo = 7.0;
int encontrado = 0;
for (int i = 0; i < n; i++) {
    if (v[i] == alvo) {
        encontrado = 1;
        printf("Encontrado na posicao %d\n", i);
    }
}
if (!encontrado)
    printf("Nao encontrado\n");
```

A busca percorre todos os elementos verificando um a um. É simples e sempre funciona — para vetores ordenados existem métodos mais eficientes, mas a busca linear é o ponto de partida natural.

5.5 Contagem condicional

```
float media = /* calculada antes */;
int acima = 0;
for (int i = 0; i < n; i++) {
    if (v[i] > media)
        acima++;
}
printf("%d elementos acima da media\n", acima);
```

Este padrão — contar elementos que satisfazem uma condição — é exatamente o que a Aula 5 não conseguia fazer, porque exige que os dados ainda estejam disponíveis **depois** de calcular a média. Com o vetor, o segundo percurso é trivial.

6 Inicialização de vetores

Assim como variáveis simples, elementos de vetor não inicializados contêm lixo. Há três formas comuns de inicializar:

```
/* 1. Lista de valores – inicializa cada elemento */
int v[5] = {10, 20, 30, 40, 50};

/* 2. Inicializacao parcial – elementos restantes recebem zero */
int v[5] = {10, 20}; /* v[2], v[3], v[4] valem 0 */

/* 3. Zerar tudo de uma vez */
int v[5] = {0}; /* todos os elementos valem 0 */
```

Quando o tamanho é omitido na declaração com lista, o compilador o deduz automaticamente:

```
int v[] = {3, 1, 4, 1, 5}; /* compilador infere tamanho 5 */
```

Atenção

Nunca use um elemento de vetor sem inicializá-lo antes. O valor presente é o que estiver naquele endereço de memória — qualquer coisa. Diferente de um ponteiro não-inicializado (que pode

causar falha imediata), um vetor com lixo pode produzir resultados incorretos silenciosamente — o tipo de bug mais difícil de encontrar.

7 Limites de índice

C não verifica automaticamente se um índice está dentro dos limites do vetor. Acessar `v[n]` ou `v[-1]` não gera um erro de compilação — e frequentemente não gera nem um erro em tempo de execução. O programa simplesmente lê ou escreve na posição de memória correspondente, seja lá o que estiver lá.

```
int v[5] = {0};
v[5] = 99;    /* acesso fora dos limites – comportamento indefinido */
v[-1] = 99;   /* idem */
```

Atenção

Acesso fora dos limites é um dos bugs mais perigosos em C. O programa pode parecer funcionar corretamente, corromper dados de outra variável, travar com **segmentation fault**, ou produzir resultados diferentes a cada execução. O compilador não avisa. A responsabilidade de manter o índice dentro dos limites é inteiramente do programador.

A proteção mais simples é nunca usar constantes numéricas como tamanho — definir o tamanho com uma constante nomeada e usá-la em todos os laços:

```
#define N 100

float notas[N];
for (int i = 0; i < N; i++) {
    scanf("%f", &notas[i]);
}
```

Se o tamanho precisar mudar, basta alterar o `#define` — todos os laços se ajustam automaticamente.

8 Vetores e funções

Na Aula 7, vimos que funções recebem **cópias** dos argumentos — passagem por valor. Isso muda com vetores.

Quando passamos um vetor para uma função, o que é copiado é o **endereço do primeiro elemento** — não todos os dados. A função recebe um ponteiro e acessa a memória original. Isso tem duas consequências:

1. **A função pode modificar o vetor original** — diferente de variáveis simples.
2. **A função não sabe o tamanho do vetor** — o endereço não carrega essa informação. O tamanho deve ser passado como parâmetro separado.

```
/* O parametro 'float v[]' e equivalente a 'float *v' */
void imprime(float v[], int n) {
    for (int i = 0; i < n; i++)
        printf("%.2f ", v[i]);
    printf("\n");
}

void zera(float v[], int n) {
```

```
    for (int i = 0; i < n; i++)
        v[i] = 0; /* modifica o vetor original – nao uma copia */
}

int main() {
    float notas[5] = {7.5, 8.0, 6.5, 9.0, 7.0};
    imprime(notas, 5); /* passa o endereco de notas[0] */
    zera(notas, 5);    /* modifica notas diretamente */
    imprime(notas, 5); /* imprime cinco zeros */
    return 0;
}
```

A sintaxe `float v[]` no parâmetro de uma função é apenas outra forma de escrever `float *v` — ambas significam “um ponteiro para float”. O compilador trata as duas formas de modo idêntico. Usar `v[]` é mais expressivo quando queremos deixar claro que o parâmetro representa um vetor; usar `*v` é mais preciso sobre o que realmente acontece. Nas aulas seguintes, as duas formas aparecerão indistintamente.

9 Exemplo integrador: notas acima da média

Voltamos ao problema da seção 1 — agora com todas as ferramentas necessárias. O programa lê n notas, calcula a média e lista quais notas estão acima dela.

```
#include <stdio.h>
#define MAX 200

/*
 * Descricao: Le n notas, calcula a media e lista as notas acima dela.
 * Entrada:  n (inteiro, 1 <= n <= MAX), seguido de n reais
 * Saida:    media e lista das notas acima da media com suas posicoes
 */

/* Preenche o vetor com n valores lidos do teclado */
void le_notas(float v[], int n) {
    for (int i = 0; i < n; i++) {
        printf("Nota %d: ", i + 1);
        scanf("%f", &v[i]);
    }
}

/* Retorna a media dos n primeiros elementos de v */
float media(float v[], int n) {
    float soma = 0;
    for (int i = 0; i < n; i++)
        soma += v[i];
    return soma / n;
}

/* Imprime os elementos de v maiores que limite, com seus indices */
void imprime_acima(float v[], int n, float limite) {
    int count = 0;
    for (int i = 0; i < n; i++) {
        if (v[i] > limite) {
            printf(" Nota %d: %.1f\n", i + 1, v[i]);
        }
    }
}
```

```
        count++;
    }
}
if (count == 0)
    printf("  Nenhuma nota acima da media.\n");
}

int main() {
    float notas[MAX];
    int n;

    printf("Quantos alunos? ");
    scanf("%d", &n);

    le_notas(notas, n);

    float med = media(notas, n);
    printf("\nMedia da turma: %.2f\n", med);
    printf("Notas acima da media:\n");
    imprime_acima(notas, n, med);

    return 0;
}
```

Este programa faz algo que era **impossível** na Aula 5: percorre os dados duas vezes — uma para calcular a média, outra para compará-la com cada elemento. O vetor é o que torna o segundo percurso possível, porque os dados continuam na memória após o primeiro laço.

Observe também a organização: cada função tem uma única responsabilidade. `main` conta a história em alto nível; os detalhes ficam encapsulados. Isso é o que aprendemos na Aula 7 — agora aplicado a dados coletivos.

10 O que vetores permitem que laços sozinhos não permitem

Vale tornar explícita a diferença entre os dois mundos:

Laço sem vetor (Aula 5)	Laço com vetor (Aula 9)
Cada valor é processado e descartado	Todos os valores ficam disponíveis após a leitura
Um único percurso sobre os dados	Múltiplos percursos possíveis
Não pode comparar um elemento com um resultado que depende de todos	Pode calcular a média e depois comparar cada nota com ela
Adequado quando cada valor é independente dos demais	Necessário quando o processamento exige ver todos os dados

A regra prática: se a resposta que você precisa depende de **todos** os valores antes de poder ser calculada, você precisa de um vetor.

11 Exercícios

1. Escreva uma função `void inverte(int v[], int n)` que inverte os elementos de um vetor **no próprio vetor** (sem usar um vetor auxiliar). Teste com {1, 2, 3, 4, 5} — resultado esperado: {5, 4, 3, 2, 1}. (Dica: troque $v[0]$ com $v[n-1]$, depois $v[1]$ com $v[n-2]$, e assim por diante. *Quantas trocas são necessárias?*)
2. Escreva uma função `int esta_ordenado(int v[], int n)` que retorna 1 se o vetor estiver em ordem não-decrescente e 0 caso contrário. Teste com vetores ordenados e não-ordenados.
3. Escreva um programa completo que leia um vetor de n inteiros e imprima:
 - A soma e a média dos elementos;
 - O maior e o menor valor e suas posições;
 - Quantos elementos são maiores que a média.
4. Escreva uma função `void copia(int origem[], int destino[], int n)` que copia os elementos de origem para destino. Verifique que modificar destino após a cópia não altera origem. Por que isso funciona diferente de copiar um ponteiro?
5. (Desafio) Escreva uma função `int busca_binaria(int v[], int n, int alvo)` que encontra alvo num vetor **ordenado** em ordem crescente usando busca binária: compare alvo com o elemento do meio; se for menor, busque na metade esquerda; se for maior, na metade direita; se igual, retorne o índice. Se não encontrar, retorne -1. Compare o número de comparações necessárias com a busca linear para $n = 1000$ no pior caso.